

Supplementary material for the work: “Thermodynamic and structural properties of the thin radiating surface layer of a white dwarf”

Riccardo Fantoni*

Università di Trieste, Dipartimento di Fisica, strada Costiera 11, 34151 Grignano (Trieste), Italy
(Dated: September 19, 2026)

Supplementary material to the article “Thermodynamic and structural properties of the thin radiating surface layer of a white dwarf”.

Keywords: White dwarf;Jellium;Path Integral;Monte Carlo;General Relativity;Thin surface layer;Thermodynamics;Structure

I. INTRODUCTION

We here report the FORTRAN code listing of the code used in the simulations developed in the publication “Thermodynamic and structural properties of the thin radiating surface layer of a white dwarf”. This is shown in Appendix A.

ACKNOWLEDGMENTS

I would like to thank prof. Saverio Moroni for his support in the development of the brownian bridge and the consequent particles permutation sampling.

Appendix A: The code

This is the code used for the RPIMC computer experiment. We list here the main FORTRAN code `rpimc-wd-rs.f` with the included `mc-wd.par` parameters file. The input simulation configuration file `data-gr-rs.in` that is read at the beginning of the run. The subroutine `ifdm.f` to build the free particle density matrix used to enforce the restriction in the RPIMC algorithm. And the `Makefile`.

```
*****
*** Makefile *****
*****

OBJ = rpimc-wd-rs.o ifdm.o

.f.o:
gfortran -c $<

rpimc-wd-rs.o: mc-wd.par rpimc-wd-rs.f
gfortran -c -freal-8-real-16 rpimc-wd-rs.f

ifdm.o: ifdm.f
gfortran -c ifdm.f

rpimc: $(OBJ)
gfortran -o rpimc $(OBJ)

clean:
- rm -f *.o *~

*****
*** rpimc-wd-rs.f *****
*****
```

```
PROGRAM PIMC
IMPLICIT NONE
*****
** MONTE CARLO SIMULATION PROGRAM FOR RESTRICTED PATH INTEGRAL **
**
** in the atmosphere of tickness DELTA of a white dwarf of
** radius RADIUS and mass MASS
**
**
** space dimensions: DIM=3
** number of particles: NP
** number of timesteps: FTNO
** units: hbar=k_B=1
** BETA=1/TEMP=FTNO*LS (LS imaginary time spacing)
**
** OBSERVABLES:
** kinetic energy KE
** potential energy V
** forces virial W
** superfluid fraction SF
** radial distribution function on 'rdf.dat'
*****
INCLUDE 'mc-wd.par' ! parameters

INTEGER*4 SEED
INTEGER*8 IDIM, STEP, I, J, K
INTEGER*8 CI, CP, IP, KP, NIP, MKP, LL, OUT1, OUT2, PIP
INTEGER*8 INIT, NSTEP, NS, IPRINT, ISAVE, IRATIO, IEQUI
INTEGER*8 MBMM, MCM, PREV(MNP)
INTEGER*8 RS, CYC(MNP), NSWAP

REAL*8 TEMP, BETA, DENSJ, v1,v2,w1,w2
REAL*8 DRMAX(MDIM), ALPHA(MDIM), CDIM
REAL*8 RANF, DUMMY, SR9, SR3
REAL*8 VLRC, VLRC6, VLRC12, WLRC, WLRC6, WLRC12
REAL*8 ACM, ACATMA, ACATMAS, ACATMAB
REAL*8 V, VNEW, VOLD, VEND, DELTV, VN, VS
REAL*8 W, WNEW, WOLD, WEND, DELTW, WN, WS
REAL*8 KE, KENEW, KEOLD, DELTKE, KEEND
REAL*8 AVV, ACV, ACVSQ, FLV
REAL*8 AVW, ACW, ACWSQ, FLW
REAL*8 AVKE, ACKE, ACKESQ, FLKE
REAL*8 ACT, ACTNEW, ACTOLD, DELTACT, DELTACTB
REAL*8 RXIOLD(MDIM), RXINEW(MDIM), RO(MDIM,0:N,MNP)
REAL*8 RXP(MDIM,0:N), RXPP(MDIM,0:N)
REAL*8 STDR, STDA, PS, KKK, VVV, WWW
REAL*8 RATIO, RATIOS, RATIOB
REAL*8 WIND, ACWIND, SFI, SF, IC
REAL*8 GA, IFDMS, EPSILON

INTEGER*8 BIN, MAXBIN, MAXBIN2, HIST(1000000), ICALOG
REAL*8 CONST, RLOWER, RUPPER
REAL*8 TSTEP, NIDEAL, GR

CHARACTER CNFILE*30, POTK*10

LOGICAL OVRLAP, IFB, IFDISP, IFBRIDGE

C PI = ACOS(-1.d0)

OPEN(UNIT = 16, FILE = 'back.dat', STATUS = 'UNKNOWN')
OPEN(UNIT = 15, FILE = 'rdf.dat', STATUS = 'UNKNOWN')
EPSILON = 0.

*****
** READ INPUT DATA **
*****

WRITE(*,(' ***** PROGRAM PIMC ***** '))
WRITE(*,(' PLANE WAVES '))
WRITE(*,(' PATH INTEGRAL MONTE CARLO PROGRAM '))
OPEN (UNIT=10, FILE='data-gr-rs.in', STATUS='UNKNOWN')
READ (10,*) I
READ (10,*) DIM
READ (10,*) I
READ (10,*) SEED
READ (10,*) I
READ (10,*) IFB
READ (10,*) I
READ (10,*) NP
READ (10,*) I
READ (10,*) (A') POTK
READ (10,*) I
READ (10,*) FTNO
READ (10,*) I
READ (10,*) FTM
```

* riccardo.fantoni@scuola.istruzione.it

```

READ (10,*) I
READ (10,*) NSTEP
READ (10,*) I
READ (10,*) IPPRINT
READ (10,*) I
READ (10,*) ISAVE
READ (10,*) I
READ (10,*) IEQUI
READ (10,*) I
READ (10,*) IRATIO
READ (10,*) I
READ (10,*) ICALCG
READ (10,*) I
READ (10,*) CNFILE
READ (10,*) I
READ (10,*) INIT
READ (10,*) I
READ (10,*) RADIUS
READ (10,*) I
READ (10,*) MASS
READ (10,*) I
READ (10,*) DELTA
READ (10,*) I
READ (10,*) ANG
READ (10,*) I
READ (10,*) TEMP
READ (10,*) I
READ (10,*) (ALPHA(J),J=1,DIM)
READ (10,*) I
READ (10,*) MBMM
READ (10,*) I
READ (10,*) RCUT
READ (10,*) I
READ (10,*) SIG
READ (10,*) I
READ (10,*) EPSR
READ (10,*) I
READ (10,*) EPSA
READ (10,*) I
READ (10,*) EPS
READ (10,*) I
READ (10,*) IFDISP
READ (10,*) I
READ (10,*) IFBRIDGE
CLOSE (UNIT=10)
WRITE(*, '( IN DIMENSION (1,2,3,...)''I12/)' ) DIM
WRITE(*, '( ***** )' )

GOTO 1111 ! comment if input from keyboard

WRITE(*, '( ***** PROGRAM PIMC ***** )' )
WRITE(*, '( PLANE WAVES )' )
WRITE(*, '( PATH INTEGRAL MONTE CARLO PROGRAM )' )

WRITE(*, '( ENTER THE SPATIAL DIMENSIONS )' )
READ (*,*) DIM
WRITE(*, '( ENTER SEED FOR RANDOM SEQUENCE )' )
READ (*,*) SEED
WRITE(*, '( IF BOSE (T/F) )' )
READ (*,*) IFB
WRITE(*, '( ENTER THE NUMBER OF PARTICLES < MNP )' )
READ (*,*) NP
WRITE(*, '( ENTER TYPE OF PAIR POTENTIAL (Many Body) )' )
READ (*,*) POTK
WRITE(*, '( ENTER THE NUMBER OF DISCRETIZATIONS < N )' )
READ (*,*) FTNO
WRITE(*, '( ENTER THE BARE MASS )' )
READ (*,*) FTM
WRITE(*, '( ENTER NUMBER OF CYCLES )' )
READ (*,*) NSTEP
WRITE(*, '( ENTER NUMBER OF STEPS BETWEEN OUTPUT LINES )' )
READ (*,*) IPPRINT
WRITE(*, '( ENTER NUMBER OF STEPS BETWEEN DATA SAVES )' )
READ (*,*) ISAVE
WRITE(*, '( ENTER NUMBER OF STEPS FOR EQUILIBRATION )' )
READ (*,*) IEQUI
WRITE(*, '( ENTER INTERVAL FOR UPDATE OF MAX. DISPL. )' )
READ (*,*) IRATIO
WRITE(*, '( ENTER INTERVAL FOR RDF CALCULATION )' )
READ (*,*) ICALCG
WRITE(*, '( ENTER THE CONFIGURATION FILE NAME )' )
READ (*,*) CNFILE
WRITE(*, '( ENTER 0 INIT Z, 1 INIT, 2 REST C, 2 REST F )' )
READ (*,*) INIT
WRITE(*, '( ENTER THE WD RADIUS )' )
READ (*,*) RADIUS
WRITE(*, '( ENTER THE WD MASS )' )
READ (*,*) MASS
WRITE(*, '( ENTER THE WD ATMOSPHERE THICKNESS )' )
READ (*,*) DELTA
WRITE(*, '( ENTER THE WD ANGULAR EXTENT OF THE PATCH )' )
READ (*,*) ANG
WRITE(*, '( ENTER THE TEMPERATURE )' )
READ (*,*) TEMP
WRITE(*, '( ENTER MAX. DISPLACEMENT DRMAX/SIGMA )' )
READ (*,*) (ALPHA(J),J=1,DIM)
WRITE(*, '( ENTER MAXIMUM NUMBER OF BRIDGE TIMESLICES )' )
READ (*,*) MBMM
WRITE(*, '( ENTER THE POTENTIAL CUTOFF DISTANCE ( 2.5 )' )' )
READ (*,*) RCUT
WRITE(*, '( ENTER SIG )' )
READ (*,*) SIG
WRITE(*, '( ENTER EPSR )' )
READ (*,*) EPSR
WRITE(*, '( ENTER EPSA )' )
READ (*,*) EPSA
WRITE(*, '( ENTER EPS )' )
READ (*,*) EPS
WRITE(*, '( IF DISPLACEMENT MOVE (T/F) )' )
READ (*,*) IFDISP
WRITE(*, '( IF BRIDGE MOVE (T/F) )' )

READ (*,*) IFBRIDGE
WRITE(*, '( IN DIMENSION (1,2,3,...)''I12/)' ) DIM
WRITE(*, '( ***** )' )

1111 CONTINUE

IF (MBMM.GT. FTNO) THEN
  WRITE(*,*) "number of timeslices in bridge > ",FTNO
  STOP
ENDIF

IF (DIM.NE. 3) THEN
  WRITE(*,*) "DIM not equal to 3"
  STOP
ENDIF

C ** INITIALIZE RANDOM NUMBER GENERATOR **

CALL SRAND ( SEED )
IF (SEED.EQ.1) THEN
  CALL SRAND ( 0 )
ENDIF

C ** DENSITY **

DENS = NP/(4.0*PI*RADIUS**2.*DELTA)/(2.*SIN(ANG/2.)*ANG)

C ** INVERSE TEMPERATURE BETA **

BETA = (1.0/TEMP)

C ** IMAGINARY TIMESTEP **

LS = BETA/FTNO ! time step
STDR = (LS/FTM)**0.5 ! standard deviation of free-particle
STDA = STDR/RADIUS

C ** MOMENT OF INERTIA OF A UNITARY SPHERICAL SHELL **

IC = 2*(NP*FTM)/3

C ** WRITE INPUT DATA **

WRITE(*, '( SEED )' ) SEED
WRITE(*, '( POTENTIAL )' ) POTK
WRITE(*, '( DIMENSIONS )' ) DIM
WRITE(*, '( NUMBER OF PARTICLES )' ) NP
WRITE(*, '( NUMBER OF CYCLES )' ) NSTEP
WRITE(*, '( NUMBER OF EQUIL. STEPS )' ) IEQUI
WRITE(*, '( OUTPUT FREQUENCY )' ) IPPRINT
WRITE(*, '( SAVE FREQUENCY )' ) ISAVE
WRITE(*, '( RATIO UPDATE FREQUENCY )' ) IRATIO
WRITE(*, '( CONFIGURATION FILE NAME )' ) CNFILE
WRITE(*, '( TEMPERATURE )' ) TEMP
WRITE(*, '( DENSITY )' ) DENS
WRITE(*, '( PARTICLE MASS )' ) FTM
WRITE(*, '( # TIME SLICES )' ) FTNO
WRITE(*, '( TIME STEP )' ) LS

C ** CONVERT INPUT DATA TO PROGRAM UNITS **

SIGMA = DELTA
SIGM(1) = ANG
SIGM(2) = ANG
DRMAX(1) = ALPHA(1) * SIGMA
DRMAX(2) = ALPHA(2) * SIGM(1)
DRMAX(3) = ALPHA(3) * SIGM(2)

IF (POTK.EQ. 'LJ') THEN
  DENSLJ = DENS * SIG ** DBLE( DIM )
  RCUT = SIGMA/2.0/SIG
ENDIF

MAXBIN = 1000
DELR = DELTA/MAXBIN
MAXBIN2 = INT ( MAXBIN/2.0 )
DO I = 1, MAXBIN
  HIST(I) = 0
ENDDO

IF (DIM.EQ. 3) CONST = 4.0 * PI * DENS / 3.0
IF (DIM.EQ. 2) CONST = PI * DENS
IF (DIM.EQ. 1) CONST = 2.0 * DENS

C ** INITIALIZE CONFIGURATION **

DO I = 1, NP
  NEXT(I) = I
  LEXT(I) = I
  DO J = 1, FTNO
    NEXT(J,I) = I
  ENDDO
ENDDO

C ** ZERO ACCUMULATORS **

ACV = 0.0
ACVSQ = 0.0
FLV = 0.0
ACW = 0.0
ACWSQ = 0.0
FLW = 0.0
ACKE = 0.0
ACKESQ = 0.0
FLKE = 0.0
ACWIND = 0.0
ACM = 0.0
ACATMA = 0.0
ACATMAB = 0.0
ACATMAS = 0.0

IF ( INIT.EQ. 0 ) THEN

```

	PRINT*, "PATHS INITIALIZATION"		keold=keold+kkk
	RX(1,:)=RADIUS-DELTA		enddo
	RX(2,:)=0.		endif
	RX(3,:)=0.		enddo
	NS = NSTEP		
	OPEN (UNIT = 9, STATUS ='UNKNOWN')	c	bridge move
	ELSEIF (INIT .EQ. 1) THEN		if(cp+mkm.le.ftn0)then
	PRINT*, "PATHS INITIALIZATION"		call bridge(rp(: ,cp,ip),rp(: ,mcm,kp),stdr,stda,
C	** READ INITIAL CONFIGURATION **	:	rxpp,out1)
	CALL INITCONWD (CNFILE) ! random configuration on radius		if (ip.ne.kp) then
	CALL READCN (CNFILE)		call bridge(rp(: ,cp,kp),rp(: ,mcm,ip),stdr,stda,
	NS = NSTEP	:	rxpp,out2)
	OPEN (UNIT = 9, STATUS ='UNKNOWN')		endif
	ELSEIF (INIT .EQ. 2) THEN		else
	PRINT*, "RESTART FROM CONFIG"		call bridge(rp(: ,cp,ip),rp(: ,mcm,nkp),stdr,stda,
C	** READ INITIAL CONFIGURATION **	:	rxpp,out1)
	CALL READCN (CNFILE)		if (ip.ne.kp) then
	NS = NSTEP		call bridge(rp(: ,cp,kp),rp(: ,mcm,nip),stdr,stda,
	OPEN (UNIT = 9, STATUS ='OLD', ACCESS ='APPEND')	:	rxpp,out2)
	ELSEIF (INIT .EQ. 3) THEN		endif
	PRINT*, "RESTART FULL"		endif
	CALL READCN (CNFILE)		
	ACATMA = 0.0		if(out1.eq.3.or.out2.eq.3) then
	ACATMAB = 0.0		out1 = 0
	ACATMAS = 0.0		out2 = 0
	READ(16,*)(HIST(I),I=1,MAXBIN),		goto 1235
&	ACV,ACVSQ,ACW,ACWSQ,ACKE,ACKESQ,ACWINW,ACM,NS		endif
	NS=NS+NSTEP		
	OPEN (UNIT = 9, STATUS ='OLD', ACCESS ='APPEND')	c	execute the bridge move
	ELSE		acatmab = acatmab + 1.d0
	WRITE (*,*) 'INIT different from 0,1,2,3'		call updaten(cp,rxpp,ip,nkp)
	STOP		if (ip.ne.kp) then
	ENDIF		call updaten(cp,rxpp,kp,nip)
C	** CALCULATE LONG RANGE CORRECTIONS **		call swapn(cp,ip,nip,kp,nkp)
C	** SPECIFIC TO THE LENNARD JONES FLUID **		call switch(next(ip),next(kp))
	IF (DIM .EQ. 1) CDIM = 1		call switch(mext(cp,ip),mext(cp,kp))
	IF (DIM .EQ. 2) CDIM = PI		acatmas = acatmas + 1.d0
	IF (DIM .EQ. 3) CDIM = 2*PI		endif
			call equiv(rx,rpn)
	SR3 = - RCUT ** (- 6. + DIM)/(-6. + DIM)		call sumspot (potk, overlap, vnew, wnew)
	SR9 = - RCUT ** (- 12. + DIM)/(-12. + DIM)		deltv=wnew-vold
			deltw=wnew-wold
	VLCRC12 = 4 * EPS * CDIM * DENSLJ * NP * SR9	c	calculate the ratio of transition probabilities
	VLCRC6 = - 4 * EPS * CDIM * DENSLJ * NP * SR3		call gauss(cp,ga)
	VLCRC = VLCRC12 + VLCRC6		
	VLCRC12 = 4.0 * VLCRC12	c	inverse mapping
	VLCRC6 = 2.0 * VLCRC6		call equiv(rp,rpn)
	VLCRC = VLCRC12 + VLCRC6	c	call equiv(rpn,rx)
C	** WRITE OUT SOME USEFUL INFORMATION **		call mappingi()
	WRITE(*,'(' SIGMA '(E10.4)')')		kenew=0.d0
	WRITE(*,'(' MAXIMUM DISPLACEMENT '(E10.4)') DRMAX(1)		do k=1,np
			if(k.eq.ip.or.k.eq.kp.or.k.eq.nip.or.k.eq.nkp)then
C	** CALCULATE INITIAL ENERGY AND CHECK FOR OVERLAPS **		do i=1,ftn0
	CALL SUMUP (POTK, OVLAP, KE, V, W)	:	call kkenergy (rx(: ,i,k), k, i,
			kkk)
	IF (POTK .EQ. 'LJ') THEN		kenew=kenew+kkk
	VS = (V + VLCRC)		enddo
	WS = (W + VLCRC)		endif
	ELSE		enddo
	VS = V - W		deltke=kenew-keold
	WS = W	c	calculate the acceptance probability
	ENDIF		call acc_p(deltv,deltke,ps)
		c	ps=ps*ga
	WRITE(*,'(' INITIAL V '(E10.4)') V /FTNO	c	accept/reject
	WRITE(*,'(' INITIAL V - W '(E10.4)') VS/FTNO		if (ps.gt.ranf(dummy)) then
	WRITE(*,'(' INITIAL W '(E10.4)') WS/FTNO	c	restricted path integral
	WRITE(*,'(' INITIAL KE '(E10.4)') KE/FTNO		if(.not.ifb)goto 41
			do i=1,ftn0
	WRITE(*,'(' START OF MARKOV CHAIN '))))		call ifdmn(rs,i,ifdms)
	WRITE(*,'(' MOVE RATIO ACTION '))))		if (ifdms .lt. epsilon) goto 1234
			endif
C	*****	41	continue
C	** LOOPS OVER ALL CYCLES AND ALL TIME SLICES **		
C	*****		ke=ke+deltke
			w=w+deltw
	DO 100 STEP = NS-NSTEP+1, NS		v=v+deltv
	CP = INT(FTNO*РАНF(DUMMY))+1 ! select a random timeslice		goto 1235
	RS = INT(FTNO*РАНF(DUMMY))+1 ! select a random referenceslice		endif
c	RS = 1	1234	continue
	MBM = INT((MBMM-1)*РАНF(DUMMY))+2 ! # timeslices in bridge		
	IP = INT(NP*РАНF(DUMMY))+1 ! select a particle		call equiv(rx,r0)
	KP = INT(NP*РАНF(DUMMY))+1 ! select another particle		acatmab = acatmab - 1.d0
	IF (.NOT.IFB) KP = IP ! for boltzmann statistics		if (ip.ne.kp) then
	IF (IFDISP) GOTO 77 ! uncomment if only displacement move		call switch(next(ip),next(kp))
			call switch(mext(cp,ip),mext(cp,kp))
C	*****		acatmas = acatmas - 1.d0
C	** BRIDGE&SWAP MOVE (BOSE STATISTICS) **		endif
C	*****	1235	continue
C	direct mapping from white dwarf atmosphere rx to plane rp	c	build the permutations cycles in lext
	call equiv(r0,rx)		call permcycc()
	call mappingd()	c	count number of swaps of two particles
	call equiv(rpn,rp)		call cycles(cyc,nsmap)
	mcm = cp+mkm-floor((cp+mkm-.1)/ftn0)*ftn0	c	balance ke between bridge and displace
	nip=next(ip)	c	call sumupke (ke)
	nkp=next(kp)	c	call sumspot (potk, overlap, v, w)
	call sumspot (potk, overlap, vold, wold)		IF (STEP.GT.IEQUI) THEN
			ACM = ACM + 1.0
	keold=0.d0		
	do k=1,np	C	** CALCULATE INSTANTANEOUS VALUES **
	if(k.eq.ip.or.k.eq.kp.or.k.eq.nip.or.k.eq.nkp)then		IF (POTK .EQ. 'LJ') THEN
	do i=1,ftn0		VN = (V + VLCRC)
	call kkenergy (rx(: ,i,k), k, i,		ELSE
	kkk)		VN = V - W

```

ENDIF
CALL CWIND( WIND )
IF (WIND.GT.3*BETA*IC*2/FTM**2) WIND=3*BETA*IC*2/FTM**2

C ** ACCUMULATE AVERAGES **
ACV = ACV + VN
ACVSQ = ACVSQ + VN*VN
ACW = ACW + WN
ACWSQ = ACWSQ + WN*WN
ACKE = ACKE + KE
ACKESQ = ACKESQ + KE*KE
ACWIND = ACWIND + WIND
ENDIF

IF (IFBRIDGE) GOTO 97

C *****
C ** ENDS BRIDGE&SWAP MOVE
C *****

77 CONTINUE

C *****
C ** DISPLACEMENT MOVE
C *****

C ** PREVIOUS IP **
DO I=1,NP
J=NEXT(I)
PREV(J)=I
ENDDO

NIP = NEXT(IP)
PIP = PREV(IP)

DO 90 C1 = 1, FTNO
DO IDIM = 1, DIM
RXIOLD(IDIM) = RX(IDIM,C1,IP)
ENDDO

C ** CALCULATE THE ENERGY OF I IN THE OLD CONFIGURATION **
CALL PENERGY ( POTK, RXIOLD, IP, C1,
: VOLD, WOLD )
CALL KENERGY ( RXIOLD, IP, C1,
: KEOLD )

C ** INSTANTANEOUS VALUE OF THE ACTION **
ACTOLD = KEOLD + VOLD

C ** MOVE I AND PICKUP THE CENTRAL IMAGE **
IDIM = 1
RXINew(IDIM) = RXIOLD(IDIM) +
: ( 2.0 * RANF ( DUMMY ) - 1.0 )*DRMAX(1)
DO IDIM = 2, DIM
RXINew(IDIM) = RXIOLD(IDIM) +
: ( 2.0 * RANF ( DUMMY ) - 1.0 )*DRMAX(IDIM)
RXINew(IDIM) = RXINew(IDIM) -
: ANINT ( RXINew(IDIM)/SIGM(IDIM-1) )*SIGM(IDIM-1)
ENDDO

C ** CONSIDER ONLY THE THIN WHITE DWRF ATMOSPHERE **
IF (RXINew(1).GT.RADIUS.OR.RXINew(1).LT.RADIUS-DELTA) THEN
GOTO 1238
ENDIF

C ** CALCULATE THE ENERGY OF I IN THE NEW CONFIGURATION **
CALL PENERGY ( POTK, RXINew, IP, C1,
: VNEW, WNEW )
CALL KENERGY ( RXINew, IP, C1,
: KENew )

C ** INSTANTANEOUS VALUE OF THE ACTION **
ACTNEW = KENew + VNEW

DELTv = VNEW - VOLD
DELTW = WNEW - WOLD
DELTKE = KENew - KEOLD

C ** CHECK FOR ACCEPTANCE **
DELTACT = ACTNEW - ACTOLD
DELTACTB = LS*DELTACT

IF ( EXP ( - DELTACTB ) .GT. RANF ( DUMMY ) ) THEN
V = V + DELTV
W = W + DELTW
KE = KE + DELTKE
ACATMA = ACATMA + 1.0

DO IDIM = 1, DIM
RX(IDIM,C1,IP) = RXINew(IDIM)
C IMAGINARY TIME PERIODIC BOUNDARY CONDITIONS
IF (C1.EQ.1) THEN
RX(IDIM,FTNO+1,PIP)=RX(IDIM,C1,IP)
ENDIF
IF (C1.EQ.FTNO) THEN
RX(IDIM,0,NIP)=RX(IDIM,C1,IP)
ENDIF
ENDDO

C RESTRICTED PATH INTEGRAL
IF(.NOT.IFB)GOTO 42
DO I=1,FTNO
CALL IFDMN(RS,I,IFDMS)
IF ( IFDMS .LT. EPSILON ) GOTO 1236
ENDDO
42 CONTINUE
GOTO 1237
1236 CONTINUE
V = V - DELTV
W = W - DELTW
KE = KE - DELTKE
ACATMA = ACATMA - 1.0

DO IDIM = 1, DIM
RX(IDIM,C1,IP) = RXIOLD(IDIM)
IF (C1.EQ.1) THEN
RX(IDIM,FTNO+1,PIP)=RX(IDIM,C1,IP)
ENDIF
IF (C1.EQ.FTNO) THEN
RX(IDIM,0,NIP)=RX(IDIM,C1,IP)
ENDIF
ENDDO
1237 CONTINUE
ENDIF
1238 CONTINUE

IF (STEP.GT.IEQUI) THEN
ACM = ACM + 1.0

C ** CALCULATE INSTANTANEOUS VALUES **
C HISTOGRAM FOR g(r)
IF ( MOD ( C1, ICALOG ) .EQ. 0 ) THEN
CALL GOFr( C1, MAXBIN, HIST, STEP )
ENDIF

IF (POTK .EQ. 'LJ') THEN
VN = ( V + VLRC )
ELSE
VN = V - W
WN = W
ENDIF
CALL CWIND( WIND )
IF (WIND.GT.3*BETA*IC*2/FTM**2) WIND=3*BETA*IC*2/FTM**2

C ** ACCUMULATE AVERAGES **
ACV = ACV + VN
ACVSQ = ACVSQ + VN*VN
ACW = ACW + WN
ACWSQ = ACWSQ + WN*WN
ACKE = ACKE + KE
ACKESQ = ACKESQ + KE*KE
ACWIND = ACWIND + WIND
ENDIF

C *****
C ** END DISPLACEMENT MOVE
C *****

90 CONTINUE

C *****
C ** ENDS LOOP OVER TIME SLICES
C *****

97 CONTINUE

C ** WRITE OUT THE INSTANTANEOUS VALUES ON FORT.9 **
SFI = WIND *FTM**2/BETA/IC/2
SF = ACWIND*FTM**2/BETA/IC/2/ACM

CALL FLUSH(9)
IF ( MOD ( STEP, IPRINT ) .EQ. 0 ) THEN
WRITE(9,*) STEP,KE/FTNO,VN/FTNO,WN/FTNO,SFI,NSWAP
ENDIF

C ** CREATE POSITION DISTRIBUTION **
CALL DISTR(30_8,STEP*FTNO*NP/30)

C ** PERFORM PERIODIC OPERATIONS **
IF ( MOD ( STEP, IRATIO ) .EQ. 0 ) THEN

C ** ADJUST MAXIMUM DISPLACEMENT **
RATIO = ACATMA / DBLE ( FTNO * IRATIO )
RATIOB = ACATMAB / DBLE ( IRATIO )
RATIOS = ACATMAS / DBLE ( IRATIO )

IF ( RATIO .GT. 0.5 ) THEN
C DRMAX(1) = DRMAX(1) * 1.05
ELSE
C DRMAX(1) = DRMAX(1) * 0.95
ENDIF
ACATMA = 0.0
ACATMAB = 0.0
ACATMAS = 0.0
ENDIF

IF ( MOD ( STEP, IPRINT ) .EQ. 0 ) THEN

```



```

        enddo
    enddo
endif
do idim=1,dim
    rpn(idim,ftn0+1,ip)=rpn(idim,1,nip)
    rpn(idim,0,nip)=rpn(idim,ftn0,ip)
enddo

return
end

subroutine swapn(j,ip,nip,kp,nkp)
implicit none
! swap the final part of the paths
INCLUDE 'mc-wd.par'

integer*8 ip,nip,kp,nkp,j,k,idim
integer*8 mcm
real*8 rr

mcm = j+mbm-floor((j+mbm-.1)/ftn0)*ftn0

if(j+mbm.le.ftn0)then
    do k=mcm,ftn0
        do idim=1,dim
            rr=rpn(idim,k,ip)
            rpn(idim,k,ip)=rpn(idim,k,kp)
            rpn(idim,k,kp)=rr
        enddo
    enddo
    do idim=1,dim
        rpn(idim,ftn0+1,ip)=rpn(idim,1,nkp)
        rpn(idim,ftn0+1,kp)=rpn(idim,1,nip)
        rpn(idim,0,nip)=rpn(idim,ftn0,kp)
        rpn(idim,0,nkp)=rpn(idim,ftn0,ip)
    enddo
endif
return
end

subroutine bridge(x0,x1,stdr,stda,xnew,out)
implicit none
! sample n gaussians with std from xnew(0)=x0 to xnew(ftn0)=x1
INCLUDE 'mc-wd.par'

integer*8 l1,l2,l3,j,idim,out
real*8 stdr,stda,d,s,xi
real*8 x0(mdim),x1(mdim),xnew(mdim,0:n)

l3=mbm

idim=1
xnew(idim,0)=x0(idim)
xnew(idim,l3)=x0(idim)+(x1(idim)-x0(idim))
do idim=2,dim
    xnew(idim,0)=x0(idim)
    xnew(idim,l3)=x0(idim)+(x1(idim)-x0(idim))-
        anint((x1(idim)-x0(idim))/sigm(idim-1))*sigm(idim-1)
enddo

do j=1,mbm-1
    l1=j-1
    l2=j
    s=stdr*(dble(l3-l2)/dble(l3-l1))*0.5d0
    idim=1
    d=xnew(idim,l3)-xnew(idim,l1)
    xnew(idim,j)=xnew(idim,l1)+d/dble(l3-l1)*xi(s)
    if(xnew(idim,j).lt.radius-delta.or.
        : xnew(idim,j).gt.radius)out=3
    s=stda*(dble(l3-l2)/dble(l3-l1))*0.5d0
    do idim=2,dim
        d=xnew(idim,l3)-xnew(idim,l1)
        d=d-anint(d/sigm(idim-1))*sigm(idim-1)
        xnew(idim,j)=xnew(idim,l1)+d/dble(l3-l1)*xi(s)
        xnew(idim,j)=xnew(idim,j)-
        : anint(xnew(idim,j)/sigm(idim-1))*sigm(idim-1)
    enddo
enddo

return
end

function xi(std)
! sample a gaussian with standard deviation std (box-muller method)
implicit none
real*8 xi,std,pi,ranf
data pi/3.14159265358979323846264338328d0/
xi=cos(pi*ranf(0.d0))*std*sqrt(-2.d0*log(tiny(pi)+ranf(0.d0)))
return
end

SUBROUTINE DISTR(NN,NORM)
IMPLICIT NONE
C WRITES ON FORT.10 THE X-POSITION DISTRIBUTION
INCLUDE 'mc-wd.par'

REAL*8 DS,DIST(0:1000)
INTEGER*8 NN,NORM,I,J,K
SAVE DIST

DS=SIGMA/NN

DO I=0,NN
    DO J=1,FTN0
        DO K=1,NP
            IF(-SIGMA/2+(I-.5)*DS.LT.RX(1,J,K).AND.
                : RX(1,J,K).LT.-SIGMA/2+(I+.5)*DS) THEN
                DIST(I)=DIST(I)+1.D0
            ENDIF
        ENDDO
    ENDDO
WRITE(10,*) -SIGMA/2+I*DS,DIST(I)/NORM
ENDDO

CLOSE(UNIT=10)

RETURN
END

FUNCTION FACT ( N )
IMPLICIT NONE
C FACTORIAL FUNCTION
INTEGER*8 FACT,N,P,I
P=1
DO I=1,N
    P=P*I
ENDDO
FACT=P
END

SUBROUTINE SUMUP (POTK,OVRLAP,KE,V,W)
IMPLICIT NONE
C *****
C ** CALCULATES THE TOTAL ENERGY **
C ** USAGE: **
C ** THE SUBROUTINE RETURNS THE TOTAL ENERGY AT THE **
C ** BEGINNING AND END OF THE RUN. **
C *****
INCLUDE 'mc-wd.par'

REAL*8 V,KE,VV,KK
LOGICAL OVRLAP
CHARACTER POTK*(*)

REAL*8 RXII,RXIJ,RVIJ,RZIJ
REAL*8 VIJ,WIJ,RIJSQ,W,WV,SQR
REAL*8 RJ(MDIM),RJO(MDIM),OMCG1,OMCG2

INTEGER*8 TAU,I,J,IDIM

C POTENTIAL ACTION

VV = 0.0
WW = 0.0

C ** LOOP OVER ALL THE PAIRS IN THE LIQUID **

DO TAU = 1, FTN0
    DO I = 1, NP
        SQR = SQRT(1.-2.*MASS/RX(1,TAU,I))
        VV = VV - LOG(ABS(RX(1,TAU,I)**2+COS(RX(2,TAU,I))/SQR))/LS
        WW = WW - LOG(ABS(RX(1,TAU,I)**2+COS(RX(2,TAU,I))/SQR))/LS
    ENDDO

    DO 100 I = 1, NP - 1
        DO 99 J = I + 1, NP

            RJ(2)=RX(2,TAU,J)
            RJ(3)=RX(3,TAU,J)
            RJO(2)=RX(2,TAU,J)
            RJO(3)=RX(3,TAU,J)
            IF (RJ(2).GT.0) THEN
                RJ(2)=RJ(2)-SIGM(1)
            ELSE
                RJ(2)=RJ(2)+SIGM(1)
            ENDIF
            IF (RJ(3).GT.0) THEN
                RJ(3)=RJ(3)-SIGM(2)
            ELSE
                RJ(3)=RJ(3)+SIGM(2)
            ENDIF
            OMCG1 = RX(2,TAU,I)*RX(2,TAU,J)+RX(3,TAU,I)*RX(3,TAU,J)
            OMCG2 = RJ(2)*RX(2,TAU,I)+RJ(3)*RX(3,TAU,I)
            IF (OMCG1.GT.OMCG2) THEN
                RX(2,TAU,J)=RJ(2)
                RX(3,TAU,J)=RJ(3)
            ENDIF

            CALL POT (TAU, I, J, RIJSQ, VIJ, WIJ, POTK)
            VV = VV + VIJ
            WW = WW + WIJ

            RX(2,TAU,J)=RJO(2)
            RX(3,TAU,J)=RJO(3)

99          CONTINUE
100         CONTINUE
        ENDDO
        V = VV
        W = WW
    ENDDO

C KINETIC ACTION

KK = 0.0

DO TAU = 1, FTN0
    DO I = 1, NP
        RXII = RX(1,TAU,I)-RX(1,TAU+1,I)
        KK=KK+FTM*RXII**2./(1.-2.*MASS/RX(1,TAU,I))
        & / (2.D0*LS**2.)

        RXII = RX(2,TAU,I)-RX(2,TAU+1,I)
        RXII = RXII - ANINT ( RXII/SIGM(1) ) *SIGM(1)
        KK=KK+FTM*(RX(1,TAU,I)*RXII)**2./(2.D0*LS**2.)
    ENDDO
ENDDO

```

```

      RXII = RX(3,TAU,I)-RX(3,TAU+1,I)
      RXII = RXII - ANINT ( RXII/SIGM(2) ) * SIGM(2)
      KK=KK+FTM*(RX(1,TAU,I)*COS(RX(2,TAU,I))*RXII)**2.
&      / (2.D0*LS**2.)
      ENDDO
    ENDDO
    KE = KK

    RETURN
  END

  SUBROUTINE SUMUPOT (POTK, OVRLAP, V, W)
  IMPLICIT NONE
  *****
  C ** CALCULATES THE TOTAL POTENTIAL ENERGY **
  C ** **
  C ** USAGE: **
  C ** **
  C ** THE SUBROUTINE RETURNS THE TOTAL ENERGY AT THE **
  C ** BEGINNING AND END OF THE RUN. **
  C *****
  INCLUDE 'mc-wd.par'

  REAL*8 V, VV, KK
  LOGICAL OVRLAP
  CHARACTER POTK*(*)

  REAL*8 RXIJ, RYIJ, RZIJ
  REAL*8 VIJ, WIJ, RIJSQ, W, WW, SQR
  REAL*8 RJ(MDIM), RJO(MDIM), OMC1, OMC2

  INTEGER*8 TAU, I, J, IDIM

  C POTENTIAL ACTION

  VV = 0.0
  WW = 0.0

  C ** LOOP OVER ALL THE PAIRS IN THE LIQUID **

  DO TAU = 1, FTWO
    DO I = 1, NP
      SQR = SQRT(1.-2.*MASS/RX(1,TAU,I))
      VV = VV - LOG(ABS(RX(1,TAU,I)**2*COS(RX(2,TAU,I))/SQR))/LS
      WW = WW - LOG(ABS(RX(1,TAU,I)**2*COS(RX(2,TAU,I))/SQR))/LS
    ENDDO

    DO 100 I = 1, NP - 1
      DO 99 J = I + 1, NP

        RJ(2)=RX(2,TAU,J)
        RJ(3)=RX(3,TAU,J)
        RJO(2)=RX(2,TAU,J)
        RJO(3)=RX(3,TAU,J)
        IF (RJ(2).GT.0) THEN
          RJ(2)=RJ(2)-SIGM(1)
        ELSE
          RJ(2)=RJ(2)+SIGM(1)
        ENDIF
        IF (RJ(3).GT.0) THEN
          RJ(3)=RJ(3)-SIGM(2)
        ELSE
          RJ(3)=RJ(3)+SIGM(2)
        ENDIF
        OMC1 = RX(2,TAU,I)*RX(2,TAU,J)+RX(3,TAU,I)*RX(3,TAU,J)
        OMC2 = RJ(2)*RX(2,TAU,I)+RJ(3)*RX(3,TAU,I)
        IF (OMC1.GT.OMC2) THEN
          RX(2,TAU,J)=RJ(2)
          RX(3,TAU,J)=RJ(3)
        ENDIF

        CALL POT (TAU, I, J, RIJSQ, VIJ, WIJ, POTK)
        VV = VV + VIJ
        WW = WW + WIJ

        RX(2,TAU,J)=RJO(2)
        RX(3,TAU,J)=RJO(3)

      99 CONTINUE
    100 CONTINUE
    V = VV
    W = WW

    RETURN
  END

  SUBROUTINE PPENERGY ( POTK, RPI, I, RPJ, J,
: TAU, V, W )
  IMPLICIT NONE
  *****
  C ** RETURNS THE POTENTIAL ENERGY OF ATOMS I AND J WITH **
  C ** ALL OTHER ATOMS PLUS THE ONE BETWEEN I AND J **
  C ** **
  C ** PRINCIPAL VARIABLES: **
  C ** **
  C ** INTEGER I, J THE ATOMS OF INTEREST **
  C ** INTEGER NP THE NUMBER OF ATOMS **
  C ** INTEGER TAU THE TIMESTEP **
  C ** REAL*8 RX, RY, RZ THE ATOM POSITIONS **
  C ** REAL*8 RXI,RYI,RZI THE COORDINATES OF ATOM I **
  C ** REAL*8 V THE POTENTIAL ENERGY OF ATOM I **
  C ** REAL*8 W THE VIRIAL OF ATOM I **
  C ** **
  C ** USAGE: **
  C ** **
  C ** THIS SUBROUTINE IS USED TO CALCULATE THE CHANGE OF ENERGY **
  C ** DURING A TRIAL MOVE OF ATOM I. IT IS CALLED BEFORE AND **
  C ** AFTER THE RANDOM DISPLACEMENT OF I. **
  C *****
  INCLUDE 'mc-wd.par'

  REAL*8 RXI(MDIM), RYI, RZI, RIJSQ, VIJ, WIJ, SQR
  INTEGER*8 I, J, TAU, IDIM
  CHARACTER POTK*(*)

  REAL*8 RXIJ, RYIJ, RZIJ, RIJSQ, VIJ, WIJ, SQR
  REAL*8 RXIO(MDIM), RJ(MDIM), RJO(MDIM), OMC1, OMC2

  C ** DURING A TRIAL MOVE OF ATOM I. IT IS CALLED BEFORE AND **
  C ** AFTER THE RANDOM DISPLACEMENT OF I. **
  C *****
  INCLUDE 'mc-wd.par'

  REAL*8 RPI(MDIM), RPJ(MDIM), RXI(MDIM), RXJ(MDIM), V, W
  INTEGER*8 I, J, K, TAU, IDIM
  CHARACTER POTK*(*)

  REAL*8 RXIJ, RYIJ, RZIJ, RIJSQ, VIJ, WIJ, SQR, SQRJ
  REAL*8 RXIO(MDIM), RXJO(MDIM)

  C MAPPING
  RXI=RPI
  RXJ=RPJ

  V = 0.0
  W = 0.0

  DO IDIM = 1, DIM
    RXIO(IDIM) = RX(IDIM,TAU,I)
    RX(IDIM,TAU,I) = RXI(IDIM)
    RXJO(IDIM) = RX(IDIM,TAU,J)
    RX(IDIM,TAU,J) = RXJ(IDIM)
  ENDDO

  SQR = SQRT(1.-2.*MASS/RX(1,TAU,I))
  SQRJ = SQRT(1.-2.*MASS/RX(1,TAU,J))
  V = V + 8*PI*SIG*DENS*
  & (RX(1,TAU,I)-(RADIUS-DELTA))**2*
  & (RX(1,TAU,I)+2*(RADIUS-DELTA))/3/RX(1,TAU,I)
  IF (I.NE.J) THEN
    V = V + 8*PI*SIG*DENS*
    & (RX(1,TAU,J)-(RADIUS-DELTA))**2*
    & (RX(1,TAU,J)+2*(RADIUS-DELTA))/3/RX(1,TAU,J)
  ENDIF
  V = V - LOG(ABS(RX(1,TAU,I)**2*COS(RX(2,TAU,I))/SQR))/LS
  W = W - LOG(ABS(RX(1,TAU,I)**2*COS(RX(2,TAU,I))/SQR))/LS

  C ** LOOP OVER ALL MOLECULES EXCEPT I AND J **

  DO 100 K = 1, NP
    IF ( K .NE. I .AND. K .NE. J ) THEN
      CALL POT (TAU, I, K, RIJSQ, VIJ, WIJ, POTK)
      V = V + VIJ
      W = W + WIJ
    ENDIF
  100 CONTINUE

  IF ( I .NE. J ) THEN
    CALL POT (TAU, J, K, RIJSQ, VIJ, WIJ, POTK)
    V = V + VIJ
    W = W + WIJ
  ENDIF

  IF ( I .NE. J ) THEN
    CALL POT (TAU, I, J, RIJSQ, VIJ, WIJ, POTK)
    V = V + VIJ
    W = W + WIJ
  ENDIF

  V = V - LOG(ABS(RX(1,TAU,J)**2*COS(RX(2,TAU,J))/SQRJ))/LS
  W = W - LOG(ABS(RX(1,TAU,J)**2*COS(RX(2,TAU,J))/SQRJ))/LS
  ENDIF

  DO IDIM = 1, DIM
    RX(IDIM,TAU,I) = RXIO(IDIM)
    RX(IDIM,TAU,J) = RXJO(IDIM)
  ENDDO

  RETURN
  END

  SUBROUTINE PENERGY ( POTK, RXI, I, TAU,
: V, W )
  IMPLICIT NONE
  *****
  C ** RETURNS THE POTENTIAL ENERGY OF ATOM I WITH ALL OTHER ATOMS. **
  C ** **
  C ** PRINCIPAL VARIABLES: **
  C ** **
  C ** INTEGER I THE ATOM OF INTEREST **
  C ** INTEGER NP THE NUMBER OF ATOMS **
  C ** INTEGER TAU THE TIMESTEP **
  C ** REAL*8 RX, RY, RZ THE ATOM POSITIONS **
  C ** REAL*8 RXI,RYI,RZI THE COORDINATES OF ATOM I **
  C ** REAL*8 V THE POTENTIAL ENERGY OF ATOM I **
  C ** REAL*8 W THE VIRIAL OF ATOM I **
  C ** **
  C ** USAGE: **
  C ** **
  C ** THIS SUBROUTINE IS USED TO CALCULATE THE CHANGE OF ENERGY **
  C ** DURING A TRIAL MOVE OF ATOM I. IT IS CALLED BEFORE AND **
  C ** AFTER THE RANDOM DISPLACEMENT OF I. **
  C *****
  INCLUDE 'mc-wd.par'

  REAL*8 RXI(MDIM), V, W
  INTEGER*8 I, J, TAU, IDIM
  CHARACTER POTK*(*)

  REAL*8 RXIJ, RYIJ, RZIJ, RIJSQ, VIJ, WIJ, SQR
  REAL*8 RXIO(MDIM), RJ(MDIM), RJO(MDIM), OMC1, OMC2

```

```

V      = 0.0
W      = 0.0

DO IDIM = 1, DIM
  RXIO(IDIM) = RX(IDIM,TAU,I)
  RX(IDIM,TAU,I) = RXI(IDIM)
ENDDO

SQR = SQRT(1.-2.*MASS/RX(1,TAU,I))
V = V - LOG(ABS(RX(1,TAU,I)**2*COS(RX(2,TAU,I))/SQR))/LS
W = W - LOG(ABS(RX(1,TAU,I)**2*COS(RX(2,TAU,I))/SQR))/LS

C ** LOOP OVER ALL MOLECULES EXCEPT I **

DO J = 1, NP
  IF ( I .NE. J ) THEN

    RJ(2)=RX(2,TAU,J)
    RJ(3)=RX(3,TAU,J)
    RJO(2)=RX(2,TAU,J)
    RJO(3)=RX(3,TAU,J)
    IF (RJ(2).GT.0) THEN
      RJ(2)=RJ(2)-SIGM(1)
    ELSE
      RJ(2)=RJ(2)+SIGM(1)
    ENDIF
    IF (RJ(3).GT.0) THEN
      RJ(3)=RJ(3)-SIGM(2)
    ELSE
      RJ(3)=RJ(3)+SIGM(2)
    ENDIF
    OMCG1 = RX(2,TAU,I)*RX(2,TAU,J)+RX(3,TAU,I)*RX(3,TAU,J)
    OMCG2 = RJ(2)*RX(2,TAU,I)+RJ(3)*RX(3,TAU,I)
    IF (OMCG1.GT.OMCG2) THEN
      RX(2,TAU,J)=RJ(2)
      RX(3,TAU,J)=RJ(3)
    ENDIF

    CALL POT (TAU, I, J, RIJSQ, VIJ, WIJ, POTK)
    V = V + VIJ
    W = W + WIJ

    RX(2,TAU,J)=RJO(2)
    RX(3,TAU,J)=RJO(3)
  ENDIF
ENDDO

DO IDIM = 1, DIM
  RX(IDIM,TAU,I) = RXIO(IDIM)
ENDDO

RETURN
END

SUBROUTINE KENERGY ( RXI, I, TAU, KE )
  IMPLICIT NONE
  *****
  ** RETURNS THE KINETIC ENERGY OF ATOM I WITH ALL OTHER ATOMS. **
  **
  ** PRINCIPAL VARIABLES:
  **
  ** INTEGER I           THE ATOM OF INTEREST
  ** INTEGER NP          THE NUMBER OF ATOMS
  ** INTEGER TAU         THE TIMESTEP
  ** REAL*8  RX, RY, RZ  THE ATOM POSITIONS
  ** REAL*8  RXI,RYI,RZI THE COORDINATES OF ATOM I
  ** REAL*8  KE          THE KINETIC ENERGY OF ATOM I
  **
  ** USAGE:
  **
  ** THIS SUBROUTINE IS USED TO CALCULATE THE CHANGE OF ENERGY
  ** DURING A TRIAL MOVE OF ATOM I. IT IS CALLED BEFORE AND
  ** AFTER THE RANDOM DISPLACEMENT OF I.
  *****
  INCLUDE 'mc-wd.par'

  REAL*8  RXI(MDIM), KE
  REAL*8  RXIS, RXII
  INTEGER*8  IP, TAU, IDIM

  KE = 0.d0

  RXIS = RX(1,TAU+1,IP)
  RXII = RXI(1) - RXIS
  KE=KE+FTM*RXII**2./(1.-2.*MASS/RXI(1))/(2.DO*LS**2.)

  RXIS = RX(2,TAU+1,IP)
  RXII = RXI(2) - RXIS
  RXII = RXII - ANINT ( RXII/SIGM(1) ) *SIGM(1)
  KE=KE+0.5*FTM*(RXI(1)*RXII/LS)**2.

  RXIS = RX(3,TAU+1,IP)
  RXII = RXI(3) - RXIS
  RXII = RXII - ANINT ( RXII/SIGM(2) ) *SIGM(2)
  KE=KE+0.5*FTM*(RXI(1)*COS(RXI(2))*RXII/LS)**2.

  RETURN
END

SUBROUTINE CWIND ( WIND )
  IMPLICIT NONE
  *****
  ** RETURNS THE WINDING NUMBER.
  *****
  INCLUDE 'mc-wd.par'

  REAL*8  WIND, WI, SURF
  REAL*8  RX11, RX21, RX31, RX12, RX22, RX32
  INTEGER*8  I, J, IDIM

  WIND = 0.d0
  SURF = 4*PI*RADIUS**2
  DO I = 1, NP
    DO J = 1, FTNO - 1
      RX11 = RX(1,J ,I)
      RX21 = RX(1,J+1,I)
      RX31 = RX(1,J+2,I)
      RX12 = RX(2,J ,I)
      RX22 = RX(2,J+1,I)
      RX32 = RX(2,J+2,I)
      WI = ((RX21-RX11)*(RX32-RX22)-(RX22-RX12)*(RX31-RX21))*
           RADIUS**2*COS(RX21)
    :   WIND = WIND + WI
    ENDDO
  ENDDO
  IF (WIND.GT.SURF) WIND = SURF
  WIND = WIND**2

  CONTINUE
  RETURN
  END

REAL*8 FUNCTION RANF ( DUMMY )
  IMPLICIT NONE
  *****
  ** RETURNS A UNIFORM RANDOM VARIATE IN THE RANGE 0 TO 1.
  **
  **
  ** *****
  **
  ** ** WARNING **
  ** *****
  **
  ** GOOD RANDOM NUMBER GENERATORS ARE MACHINE SPECIFIC.
  ** PLEASE USE THE ONE RECOMMENDED FOR YOUR MACHINE.
  **
  ** RAND(FLAG) returns a pseudo-random number from a uniform
  ** distribution between 0 and 1. If FLAG is 0, the next number
  ** in the current sequence is returned; if FLAG is 1, the
  ** generator is restarted by CALL SRAND(0); if FLAG has any
  ** other value, it is used as a new seed with SRAND.
  *****
  INTEGER*8  L, C, M
  PARAMETER ( L = 1029, C = 221591, M = 1048576 )
  INTEGER*8  SEED
  SAVE      SEED
  DATA     SEED / 0 /
  REAL*8  DUMMY

  SEED = MOD ( SEED * L + C, M )
  RANF = DBLE( SEED ) / M

```

```

      RANF = RAND ( )

      RETURN
      END

      SUBROUTINE READCN ( CNFILE )
      IMPLICIT NONE
C *****
C ** SUBROUTINE TO READ IN THE CONFIGURATION FROM UNIT 10 **
C *****
      INCLUDE 'mc-wd.par'

      CHARACTER CNFILE*(*) , HASH*1, MYFMT*77

      INTEGER*8 CNUNIT, I, J, NNP, NI, IDIM
      PARAMETER ( CNUNIT = 10 )

C *****

      OPEN ( UNIT = CNUNIT, FILE = CNFILE, STATUS = 'OLD' )
      WRITE(MYFMT,'(A,I10,A)') '(I3,3X,' ,DIM,'(D12.6,3X))'

      READ ( CNUNIT,* ) HASH, FTNO, NNP
      IF ( NNP .NE. NP ) STOP 'N ERROR IN READCN'

      DO 100 I = 1, NNP
        READ ( CNUNIT,* ) HASH, NEXT(I)
        NI = NEXT(I)
        DO 90 J = 1, FTNO+1
          READ ( CNUNIT, MYFMT ) LEXT(I), (RX(IDIM,J,I),IDIM=1,DIM)
          RX(1,J,I) = RX(1,J,I) + RADIUS - DELTA
90        ENDDO
          READ ( CNUNIT,13 )
          READ ( CNUNIT,13 )
          DO IDIM = 1, DIM
            RX(IDIM,0,NI) = RX(IDIM,FTNO,I)
          ENDDO
100      ENDDO
13      FORMAT(A2)

      CLOSE ( UNIT = CNUNIT )

      RETURN
      END

      SUBROUTINE WRITCN ( CNFILE )
      IMPLICIT NONE
C *****
C ** SUBROUTINE TO WRITE OUT THE CONFIGURATION TO UNIT 10 **
C *****
      INCLUDE 'mc-wd.par'

      CHARACTER CNFILE*(*) , MYFMT*77

      INTEGER*8 CNUNIT, I, J, IDIM
      PARAMETER ( CNUNIT = 10 )

C *****

      OPEN ( UNIT = CNUNIT, FILE = CNFILE, STATUS = 'UNKNOWN' )
      WRITE(MYFMT,'(A,I10,A)') '(I3,3X,' ,DIM,'(D12.6,3X))'

      WRITE(*,*) 'output to file -----'
      :-----
      CALL FLUSH ( CNUNIT )
      WRITE ( CNUNIT,* ) '#',FTNO,NP

      DO I = 1, NP
        WRITE ( CNUNIT,* ) '#', NEXT(I)
        DO J = 1, FTNO+1
          WRITE ( CNUNIT, MYFMT ) LEXT(I), RADIUS-RX(1,J,I),
          : (RX(IDIM,J,I),IDIM=2,DIM)
        ENDDO
        WRITE ( CNUNIT,13 ) ''
        WRITE ( CNUNIT,13 ) ''
      ENDDO
13      FORMAT(A2)

      CLOSE ( UNIT = CNUNIT )

      RETURN
      END

      SUBROUTINE INITCNSPH ( CNFILE )
      IMPLICIT NONE
C *****
C ** SUBROUTINE TO INITIALIZE THE CONFIGURATION TO UNIT 10 **
C *****
      INCLUDE 'mc-wd.par'

      REAL*8 RANF
      CHARACTER CNFILE*(*) , MYFMT*77

      INTEGER*8 CNUNIT, I, J, IDIM
      REAL*8 RXI(DIM)
      PARAMETER ( CNUNIT = 10 )

C *****

      OPEN ( UNIT = CNUNIT, FILE = CNFILE, STATUS = 'UNKNOWN' )
      WRITE(MYFMT,'(A,I10,A)') '(I3,3X,' ,DIM,'(D12.6,3X))'

      WRITE ( CNUNIT,* ) '#',FTNO,NP

      DO I = 1, NP
        WRITE ( CNUNIT,* ) '#', NEXT(I)
        RXI(1) = 0.
        RXI(2) = SIGM(2)*(RANF(0.40)-.5)
        DO J = 1, FTNO+1
          WRITE ( CNUNIT, MYFMT ) I, RXI(1), RXI(2)
          ENDDO
          WRITE ( CNUNIT,13 ) ''
          WRITE ( CNUNIT,13 ) ''
        ENDDO
13      FORMAT(A2)

      CLOSE ( UNIT = CNUNIT )

      RETURN
      END

      SUBROUTINE INITCNWD ( CNFILE )
      IMPLICIT NONE
C *****
C ** SUBROUTINE TO INITIALIZE THE CONFIGURATION TO UNIT 10 **
C *****
      INCLUDE 'mc-wd.par'

      REAL*8 RANF
      CHARACTER CNFILE*(*) , MYFMT*77

      INTEGER*8 CNUNIT, I, J, IDIM
      REAL*8 RXI(DIM)
      PARAMETER ( CNUNIT = 10 )

C *****

      OPEN ( UNIT = CNUNIT, FILE = CNFILE, STATUS = 'UNKNOWN' )
      WRITE(MYFMT,'(A,I10,A)') '(I3,3X,' ,DIM,'(D12.6,3X))'

      WRITE ( CNUNIT,* ) '#',FTNO,NP

      DO I = 1, NP
        WRITE ( CNUNIT,* ) '#', NEXT(I)
        RXI(1) = DELTA*RANF(0.40)
        RXI(2) = SIGM(1)*(RANF(0.40)-.5)
        RXI(3) = SIGM(2)*(RANF(0.40)-.5)
        DO J = 1, FTNO+1
          WRITE ( CNUNIT, MYFMT ) I, RXI(1), RXI(2), RXI(3)
          ENDDO
          WRITE ( CNUNIT,13 ) ''
          WRITE ( CNUNIT,13 ) ''
        ENDDO
13      FORMAT(A2)

      CLOSE ( UNIT = CNUNIT )

      RETURN
      END

      SUBROUTINE POT (TAU, I, J, RIJSQ, VIJ, WIJ, POTK)
      IMPLICIT NONE
C *****
C ** SUBROUTINE FOR THE PAIR POTENTIAL **
C *****
      INCLUDE 'mc-wd.par'

      CHARACTER POTK*(*)
      INTEGER*8 TAU, I, J
      REAL*8 RIJ, RIJSQ, RMIN, RMSQ, RCSQ, SR2, SR6, VIJ, WIJ
      REAL*8 COU3, QHS3, F
      REAL*8 R1I, R1J, R2I, R2J, R3I, R3J, VIJR, CG, SG, SG2
      PARAMETER ( COU3 = 4.76015472795910701328763470057d0 )
      PARAMETER ( QHS3 = 24.d0 )

C      PI = ACOS(-1.d0)

      VIJ = 0.d0
      WIJ = 0.d0
      IF (DIM .GT. 3) THEN
        WRITE(*,*) "DIM > 3 !!!"
        STOP
      ENDIF
      IF (POTK .EQ. 'FREE') RETURN

      IF (POTK .EQ. 'LJ') THEN
        CCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCC
        C W. L. McMillan, Phys. Rev. A 138, 442 (1965) C
        CCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCC
        RMIN = 2.d0**(1./6)
        C RMIN = EPSA
        C RMIN = TINY(PI)
        RMIN = 0.d0
        RIJSQ = RIJSQ / (SIG * SIG)
        RMSQ = RMIN*RMIN
        RCSQ = RCUT*RCUT
        IF (RIJSQ .LT. RMSQ) THEN
          RIJSQ = RMSQ
          SR2 = 1.d0 / RIJSQ
          SR6 = SR2 * SR2 * SR2
          VIJ = SR6 * ( SR6 - 1.d0 )
          WIJ = SR6 * ( SR6 - 5.d-1 )
          VIJ = 4.d0 * EPS * VIJ
          WIJ = 48.d0 * EPS * WIJ / 3.d0
        ELSEIF (RIJSQ .GT. RMSQ .AND. RIJSQ .LT. RCSQ) THEN
          SR2 = 1.d0 / RIJSQ
          SR6 = SR2 * SR2 * SR2
          VIJ = SR6 * ( SR6 - 1.d0 )
          WIJ = SR6 * ( SR6 - 5.d-1 )
          VIJ = 4.d0 * EPS * VIJ
          WIJ = 48.d0 * EPS * WIJ / 3.d0
        ENDIF
      ELSEIF (POTK .EQ. 'BUMP') THEN
        RIJ = SQRT( RIJSQ )
        IF (RIJ .LE. SIG) THEN
          VIJ = EPSR
        ENDIF
      ENDIF

```



```

      RX(3,J,I)=RP(3,J,I)
      ENDDO
    ENDDO

      RETURN
    END

      SUBROUTINE GAUSS(CP,GA)
      IMPLICIT NONE
      *****
      ** GAUSSIAN TRANSITION PROBABILITY FOR BROWNIAN BRIDGE **
      *****
      INCLUDE      'mc-wd.par'

      INTEGER*8 I,J,K,CP,MCM
      REAL*8 GA,GAR,GAA,VAR,VAA,DD

      MCM = CP+MBM-FLOOR((CP+MBM-.1)/FTNO)*FTNO
      VAR = 2*LS/FTM
      VAA = 2*LS/FTM/RADIUS**2.

      GAR = 0.DO
      GAA = 0.DO

      I=1
      DO J=1,NP
        IF(CP+MBM.LE.FTNO)THEN
          DO K=CP+1,MCM
            DD=RPN(I,K,J)-RPN(I,K-1,J)
            GAR=GAR+DD**2
            DD=RP(I,K,J)-RP(I,K-1,J)
            GAR=GAR-DD**2
          ENDDO
        ELSE
          DO K=CP+1,FTNO
            DD=RPN(I,K,J)-RPN(I,K-1,J)
            GAR=GAR+DD**2
            DD=RP(I,K,J)-RP(I,K-1,J)
            GAR=GAR-DD**2
          ENDDO
          DO K=1,MCM
            DD=RPN(I,K,J)-RPN(I,K-1,J)
            GAR=GAR+DD**2
            DD=RP(I,K,J)-RP(I,K-1,J)
            GAR=GAR-DD**2
          ENDDO
        ENDIF
      ENDDO

      GAR = EXP(GAR/VAR)

      DO I=2,DIM
        DO J=1,NP
          IF(CP+MBM.LE.FTNO)THEN
            DO K=CP+1,MCM
              DD=RPN(I,K,J)-RPN(I,K-1,J)
              DD=DD-ANINT ( DD/SIGM(I-1) ) *SIGM(I-1)
              GAA=GAA+DD**2
              DD=RP(I,K,J)-RP(I,K-1,J)
              DD=DD-ANINT ( DD/SIGM(I-1) ) *SIGM(I-1)
              GAA=GAA-DD**2
            ENDDO
          ELSE
            DO K=CP+1,FTNO
              DD=RPN(I,K,J)-RPN(I,K-1,J)
              DD=DD-ANINT ( DD/SIGM(I-1) ) *SIGM(I-1)
              GAA=GAA+DD**2
              DD=RP(I,K,J)-RP(I,K-1,J)
              DD=DD-ANINT ( DD/SIGM(I-1) ) *SIGM(I-1)
              GAA=GAA-DD**2
            ENDDO
          DO K=1,MCM
            DD=RPN(I,K,J)-RPN(I,K-1,J)
            DD=DD-ANINT ( DD/SIGM(I-1) ) *SIGM(I-1)
            GAA=GAA+DD**2
            DD=RP(I,K,J)-RP(I,K-1,J)
            DD=DD-ANINT ( DD/SIGM(I-1) ) *SIGM(I-1)
            GAA=GAA-DD**2
          ENDDO
        ENDIF
      ENDDO
      ENDDO

      GAA = EXP(GAA/VAA)
      GA = GAA*GAR

      RETURN
    END

      SUBROUTINE EQUIV(A,B)
      IMPLICIT NONE
      *****
      ** A = B WITH A, B TWO PATHS **
      *****
      INCLUDE      'mc-wd.par'

      INTEGER*8 I,J,K
      REAL*8 A(MDIM,0:N,MNP),B(MDIM,0:N,MNP)

      DO I=1,DIM
        DO J=0,FTNO+1
          DO K=1,NP
            A(I,J,K)=B(I,J,K)
          ENDDO
        ENDDO
      ENDDO

      RETURN
    END

```

```
c cccccccccc determines the sign of the free spinless Fermi density matrix
c dim = spatial dimensions
c nf = number of fermions
c beta = inverse temperature
c itau0= initial time slice
c itaul= final time slice
c sign = sign of the free fermion density matrix
ccccccccc subroutine ifdm(itau0,itaul,sign)
implicit none
include 'mc-wd.par'
integer*8 itau0,itaul,i,j
real*8 rrv,sum,rvec,DMSGT,epsilon,det,sign
real*8 aaa(np,np),beta,lamba

epsilon=1.d-100
beta=ls+abs(itaul-itau0)
lambda=1./2./ftm

if (itau0.eq.itaul) then
    sign=1.
    return
endif

c set up the free particle density matrix
do i=1,np
    do j=1,np
        rrv=sin(rx(1,itau0,j))*sin(rx(1,itaul,i))+
            cos(rx(1,itau0,j))*cos(rx(1,itaul,i))*
            & cos(rx(2,itau0,j)-rx(2,itaul,i))
        rvec=radius*cos(rrv)
        sum=rvec**2.
        sum=sum/(4.*beta*lambda)
        aaa(i,j)=exp(-sum)
        & aaa(i,j)=aaa(i,j)/(4*pi*beta*lambda)**(dim/2.)
    enddo
enddo

det=DMSGT(epsilon,np,aaa)

c sign of the determinant
sign=det

return
end

=====
*** ifdm.f =====
=====

ccccccccc determines the sign of the free spinless Fermi density matrix
c dim = spatial dimensions
c nf = number of fermions
c beta = inverse temperature
c itau0= initial time slice
c itaul= final time slice
c sign = sign of the free fermion density matrix
ccccccccc subroutine ifdmn(itau0,itaul,sign)
implicit none
include 'mc-wd.par'
integer*8 itau0,itaul,i,j
real*8 rrv,sum,rvec,DMSGT,epsilon,det,sign
real*8 aaa(np,np),beta,lambda

epsilon=1.d-100
beta=ls+abs(itaul-itau0)
lambda=1./2./ftm

if (itau0.eq.itaul) then
    sign=1.
    return
endif

c set up the free particle density matrix
do i=1,np
    do j=1,np
c euclidean distance
rrv=sin(rpnm(2,itau0,j))*sin(rpnm(2,itaul,i))+
        cos(rpnm(2,itau0,j))*cos(rpnm(2,itaul,i))*
        & cos(rpnm(3,itau0,j)-rpnm(3,itaul,i))
rvec=rpnm(1,itau0,i)**2.+rpnm(1,itau0,j)**2.-
        & 2.*rpnm(1,itau0,i)*rpnm(1,itau0,j)*rrv
sum=rvec
sum=sum/(4.*beta*lambda)
aaa(i,j)=exp(-sum)
& aaa(i,j)=aaa(i,j)/(4*pi*beta*lambda)**(dim/2.)
    enddo
enddo

det=DMSGT(epsilon,np,aaa)

c sign of the determinant
sign=det

return
end

!The subroutine TSGRT applies to input real square matrix A(n,n) the upper
!triangularization algorithm of Gauss method with full pivoting and keeps
!trace of successive transformations done in integer vectors KP and LP.
-----
! Input parameters:
! eps precision (real*8)
! n size of A matrix (integer)
```

```

! A      pointer to input real square matrix (real*8)
! Output parameters:
! it      flag=1 if A matrix ok, =0 if A matrix is singular (integer)
! C      pointer to table storing main diagonal elements and supra-
!        diagonal elements of upper triangular matrix and the multi-
!        plying coefficients used during triangularization process
! KP      table storing informations concerning the column exchanges
!        during process (integer)
! LP      table storing informations concerning the line exchanges
!        during process (integer)
!-----
!The table C is first initialized to A matrix, then receives at each step k
!of the triangularization process, usefull elements of A matrix at step k for
!k=1,2,...n.
!The variables po(real*8), lo and ko(integer) store respectively pivot at step k,
!its line number and its column number.
!-----

```

```

Subroutine TSRTG(eps, n, A, it, C, Kp, Lp)

```

```

real*8 eps
integer*8 n, it
real*8 A(n,n), C(n,n)
integer*8 Kp(n), Lp(n)
real*8 po, to
C=A
it=1
k=1
do while (it==1.and.k<n)
  po=C(k,k)
  lo=k
  ko=k
  do i=k, n
    do j=k, n
      if (dabs(C(i,j))>dabs(po)) then
        po=C(i,j)
        lo=i
        ko=j
      end if
    end do
  end do
  Lp(k)=lo
  Kp(k)=ko
  if (dabs(po)<eps) then
    it=0
  else
    if (lo.ne.k) then
      do j=k, n
        to=C(k,j)
        C(k,j)=C(lo,j)
        C(lo,j)=to
      end do
    end if
    if (ko.ne.k) then
      do i=1, n
        to=C(i,k)
        C(i,k)=C(i,ko)
        C(i,ko)=to
      end do
    end if
    do i=k+1, n
      C(i,k)=C(i,k)/po
      do j=k+1, n
        C(i,j)=C(i,j)-C(i,k)*C(k,j)
      end do
    end do
    k=k+1
  end if
end do
if (it==1.and.dabs(C(n,n))<eps) it=0
return
End

```

```

!The function DMGT returns the determinant of a real square matrix
!A(n,n) by Gauss method with full pivoting.
!-----

```

```

! Input parameters:
! eps      precision (real*8)
! n        size of A matrix (integer)
! A        pointer to input real square matrix
! Output parameters:
! None
!-----

```

```

!The procedure TSRTG is used to reduce A matrix to an upper triangular matrix.
!Output variables are it(integer), C(n,n), Kp(n) and Lp(n).
!If it=0, matrix A is singular, if it=1, matrix A is regular. Table C contains
!at location i,j (j>=i) the corresponding element of the upper triangular matrix.
!Tables Lp and Kp contain informations relative to exchanges of line or column
!that occurred during the process. For instance, the element number k of Lp is
!an integer <> k if an exchange of line has been made at step k (k=1,2,...n).
!The number of exchanges of lines and columns is stored in integer L. the
!determinant of A matrix is stored in d0 (real*8).
!-----

```

```

real*8 Function DMGT(eps, n, A)
integer*8 n, it, Kp(n), Lp(n)
real*8 eps, A(n,n), C(n,n)
real*8 d0

call TSRTG(eps,n,A,it,C,Kp,Lp) !call triangularization subroutine
if (it==0) then
  d0=0.d0
  !matrix singular, det=0
else
  d0=1.d0
  !matrix regular, det<>0
  do k=1, n
    d0=d0*C(k,k)
  end do
  l=0
  do k=1, n-1
    if (Lp(k).ne.k) l=l+1
    if (Kp(k).ne.k) l=l+1
  end do
  if (MOD(l,2).ne.0) d0=-d0 !l is odd

```

```

end if
DMGT=d0
return
End

```

```

*****
*** mc-wd.par *****
*****

```

```

REAL*8      PI
REAL*8      DELR
INTEGER*8    MDIM, N, MNP

```

```

PARAMETER ( PI = 3.1415926535897932384626433832840 )

```

```

PARAMETER ( MDIM = 10 ) ! maximum number of dimensions
PARAMETER ( MNP = 1000 ) ! maximum number of particles
PARAMETER ( N = 3000 ) ! maximum number of time slices

```

```

INTEGER*8    DIM
REAL*8        RX(MDIM,0:N,MNP)
REAL*8        RPN(MDIM,0:N,MNP), RP(MDIM,0:N,MNP)
REAL*8        FTM, LS, SIGMA, SIGM(MDIM)
REAL*8        MASS, RADIUS, DELTA, ANG, DENS
REAL*8        SIO, EPSA, EPSR, EPS, RCUT
INTEGER*8      FTNO, NP
INTEGER*8      NEXT(MNP), LEXT(MNP), MEXT(MNP), MBM

```

```

! path
COMMON / BLOCK1 / RX, RP, RPN, DELR, DIM
! pair-potential parameters
COMMON / BLOCK2 / SIG, EPSR, EPSA, EPS, RCUT
! mass, timestep, box edge, # timeslices, # particles
COMMON / BLOCK3 / FTM, LS, SIGMA, SIGM, FTNO, NP
! permutations
COMMON / BLOCK4 / NEXT, LEXT, MEXT, MBM
! white dwarf radius, mass, delta, density
COMMON / BLOCK5 / RADIUS, MASS, DELTA, ANG, DENS

```

```

*****
*** data-gr-rs.in *****
*****

```

```

0 number of spatial dimensions      (wd)
3
1 seed of the random sequence      RAND
4
2 if bose                          (T/F)
T
3 number of particles                (<= MNP)
30
4 the potential                      SUBROUTINE POT
WD
5 # time slices = 1/temperature/timestep  (< N)
30
6 mass                              (hbar = kb = 1)
4.2d-23
7 # of cycles                        (nstep)
5000000000000000000
8 # of steps between output lines    (iprint)
100
9 # of steps between configuration saves  (isave)
100
10 # of steps for equilibration       (iequi)
100
11 # of steps for acceptance ratios    (iratio)
100
12 # of steps for rdf calculation      (icalcg < #5)
10
13 configuration file name
conf.xyz
14 0 init zero, 1 init, 2 rest conf, 3 rest full
1
15 radius of the wd
4.3d40
16 mass of the wd
9.1d37
17 wd atmosphere thickness
4.3d23
18 wd angular extent of the patch
1.d-17
19 temperature                        THERMODYNAMICS
7.d-27
20 maximum displacement/box edge
1.d-1
1.d-1
1.d-1
21 maximum # of bridge timeslices    (> 1; <= #5)
20
22 potential cutoff distance (LJ)    SUBROUTINE POT
2.d0
23 sig (LJ 2.566)                    SUBROUTINE POT
0.00729735
24 epsr (LJ INIT)                    SUBROUTINE POT
1.d10
25 epsa                              SUBROUTINE POT
1.d0
26 eps (LJ 10.22)                    SUBROUTINE POT
1.d1
27 if only displace                  (T/F)
F
28 if only bridge                    (T/F)
F

```

AUTHOR DECLARATIONS

Data availability

The data that support the findings of this study are available from the corresponding author upon reasonable request.

Conflicts of interest

None declared.

Funding

None declared.
